内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

【面试】操作系统面试题

kuangd_1992 于 2023-05-29 22:13:07 发布

GitCode 开源社区 文章已被社区收录 加入社区

面试 专栏收录该内容 0 订阅 2 篇文章 订阅专栏

操作系统面试题一

什么是操作系统？请简要概述一下

操作系统是管理计算机硬件和软件资源的计算机程序，提供一个计算机用户与计算机硬件系统之间的接口。

向上对用户程序提供接口，向下接管硬件资源。

操作系统本质上也是一个软件，作为最接近硬件的系统软件，负责处理器管理、存储器管理、设备管理、文件管理和提供用户接口。

操作系统有哪些分类？

操作系统常规可分为批处理操作系统、分时操作系统、实时操作系统。

若一个操作系统兼顾批操作和分时的功能，则称该系统为通用操作系统。

常见的通用操作系统有：Windows、Linux、MacOS等。

什么是内核态和用户态？

为了避免操作系统和关键数据被用户程序破坏，将处理器的执行状态分为内核态和用户态。

内核态是操作系统管理程序执行时所处的状态，能够执行包含特权指令在内的一切指令，能够访问系统内所有的存储空间。

用户态是用户程序执行时处理器所处的状态，不能执行特权指令，只能访问用户地址空间。

版权声明：本文为博主原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接和本声明。
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

用户程序运行在用户态,操作系统内核运行在内核态。

如何实现内核态和用户态的切换?

处理器从用户态切换到内核态的方法有三种：系统调用、异常和外部中断。

1. 系统调用是操作系统的最小功能单位，是操作系统提供的用户接口，系统调用本身是一种软中断。
2. 异常，也叫做内中断，是由错误引起的，如文件损坏、缺页故障等。
3. 外部中断，是通过两根信号线来通知处理器外设的状态变化，是硬中断。

并发和并行的区别

1. 并发（concurrency）：指宏观上看起来两个程序在同时运行，比如说在单核cpu上的多任务。但是从微观上看两个程序的指令是交织着运行的，指令之间交错执行，在单个周期内只运行了一个指令。这种并发并不能提高计算机的性能，只能提高效率（如降低某个进程的相应时间）。
2. 并行（parallelism）：指严格物理意义上的同时运行，比如多核cpu，两个程序分别运行在两个核上，两者之间互不影响，单个周期内每个程序都运行了自己的指令，也就是运行了两条指令。这样说来并行的确提高了计算机的效率。所以现在的cpu都是往多核方面发展。

什么是进程?

进程是操作系统中最重要的抽象概念之一，是资源分配的基本单位，是独立运行的基本单位。

进程的经典定义就是一个执行中程序的实例。系统中的每个程序都运行在某个进程的上下文（context）中。

上下文是由程序正确运行所需的状态组成的。这个状态包括存放在内存中的程序的代码和数据，它的栈、通用目的寄存器的内容、程序计数器、环境变量以及打开文件描述符的集合。

进程一般由以下的部分组成：

1. 进程控制块PCB，是进程存在的唯一标志，包含进程标识符PID，进程当前状态，程序和数据地址，进程优先级、CPU现场保护区（用于进程切换），占有的资源清单等。
2. 程序段
3. 数据段

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

进程的基本操作

以Unix系统举例：

1. 进程的创建：fork()。新创建的子进程几乎但不完全与父进程相同。子进程得到与父进程用户级虚拟地址空间相同的(但是独立的)一份副本，包括代码和数据段、堆、共享库以及用户栈。子进程还获得与父进程任何打开文件描述符相同的副本，这就意味着当父进程调用 fork 时，子进程可以读写父进程中打开的任何文件。父进程和新创建的子进程之间最大的区别在于它们有不同的 PID。fork函数是有趣的（也常常令人迷惑）， 因为它只被调用一次，却会返回两次：一次是在调用进程（父进程）中，一次是在新创建的子进程中。在父进程中，fork 返回子进程的 PID。在子进程中，fork 返回 0。因为子进程的 PID 总是为非零，返回值就提供一个明 确的方法来分辨程序是在父进程还是在子进程中执行。

```
pid_t fork(void);
```

2. 回收子进程：当一个进程由于某种原因终止时，内核并不是立即把它从系统中清除。相反，进程被保持在一种已终止的状态中，直到被它的父进程回收（reaped）。当父进程回收已终止的子进程时，内核将子进程的退出状态传递给父进程，然后抛弃已终止的进程。一个进程可以通过调用 waitpid 函数来等待它的子进程终止或者停止。

```
pid_t waitpid(pid_t pid, int *statusp, int options);
```

3. 加载并运行程序：execve 函数在当前进程的上下文中加载并运行一个新程序。

```
int execve(const char *filename, const char *argv[], const char *envp[]);
```

4. 进程终止：

```
void exit(int status);
```

简述进程间通信方法

每个进程各自有不同的用户地址空间,任何一个进程的全局变量在另一个进程中都看不到，所以进程之间要交换数据必须通过内核,在内核中开辟一块缓冲区,进程A把数据从用户空间拷到内核缓冲区,进程B再从内核缓冲区把数据读走,内核提供的这种机制称为进程间通信。

不同进程间的通信本质：进程之间可以看到一份公共资源；而提供这份资源的形式或者提供者不同，造成了通信方式不同。

进程间通信主要包括管道、系统IPC（包括消息队列、信号量、信号、共享内存等）、以及套接字socket。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

进程如何通过管道进行通信

管道是一种最基本的IPC机制，作用于有血缘关系的进程之间，完成数据传递。调用pipe系统函数即可创建一个管道。有如下特质：

1. 其本质是一个伪文件(实为内核缓冲区)
2. 由两个文件描述符引用，一个表示读端，一个表示写端。
3. 规定数据从管道的写端流入管道，从读端流出。

管道的原理: 管道实为内核使用环形队列机制，借助内核缓冲区实现。

管道的局限性：

1. 数据自己读不能自己写。
2. 数据一旦被读走，便不在管道中存在，不可反复读取。
3. 由于管道采用半双工通信方式。因此，数据只能在一个方向上流动。
4. 只能在有公共祖先的进程间使用管道。

进程如何通过共享内存通信?

它使得多个进程可以访问同一块内存空间，不同进程可以及时看到对方进程中对共享内存中数据得更新。这种方式需要依靠某种同步操作，如互斥锁和信号量等。

特点：

1. 共享内存是最快的一种IPC，因为进程是直接对内存进行操作来实现通信，避免了数据在用户空间和内核空间来回拷贝。
2. 因为多个进程可以同时操作，所以需要进行同步处理。
3. 信号量和共享内存通常结合在一起使用，信号量用来同步对共享内存的访问。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

什么是信号

一个信号就是一条小消息，它通知进程系统中发生了一个某种类型的事件。 Linux 系统上支持的30 种不同类型的信号。 每种信号类型都对应于某种系统事件。低层的硬件异常是由内核异常处理程序处理的，正常情况下，对用户进程而言是不可见的。信号提供了一种机制，通知用户进程发生了这些异常。

1. 发送信号：内核通过更新目的进程上下文中的某个状态，发送（递送）一个信号给目的进程。发送信号可以有如下两种原因：
   - 内核检测到一个系统事件，比如除零错误或者子进程终止。
   - 一个进程调用了kill 函数， 显式地要求内核发送一个信号给目的进程。一个进程可以发送信号给它自己。
2. 接收信号：当目的进程被内核强迫以某种方式对信号的发送做出反应时，它就接收了信号。进程可以忽略这个信号，终止或者通过执行一个称为信号处理程序(signal handler)的用户层函数捕获这个信号。

如何编写正确且安全的信号处理函数

1. 处理程序要尽可能简单。 避免麻烦的最好方法是保持处理程序尽可能小和简单。例如，处理程序可能只是简单地设置全局标志并立即返回；所有与接收信号相关的处理都由主程序执行，它周期性地检查(并重置)这个标志。
2. 在处理程序中只调用异步信号安全的函数。 所谓异步信号安全的函数(或简称安全的函数)能够被信号处理程序安全地调用，原因有二：要么它是可重入的(例如只访问局部变量)，要么它不能被信号处理程序中断。
3. 保存和恢复errno。 许多Linux 异步信号安全的函数都会在出错返回时设置errno在处理程序中调用这样的函数可能会干扰主程序中其他依赖于分。解决方法是在进人处理程序时把errno 保存在一个局部变量中，在处理程序返回前恢复它。注意，只有在处理程序要返回时才有此必要。如果处理程序调用_exit终止该进程，那么就不需要这样做了。
4. 阻塞所有的信号，保护对共享全局数据结构的访问。 如果处理程序和主程序或其他处理程序共享一个全局数据结构，那么在访问(读或者写)该数据结构时，你的处理程序和主程序应该暂时阻塞所有的信号。这条规则的原因是从主程序访问一个数据结构d 通常需要一系列的指令，如果指令序列被访问d 的处理程序中断，那么处理程序可能会发现d 的状态不一致，得到不可预知的结果。在访问d 时暂时阻塞信号保证了处理程序不会中断该指令序列。
5. 用volatile 声明全局变量。 考虑一个处理程序和一个main 函数，它们共享一个全局变量g 。处理程序更新g，main 周期性地读g， 对于一个优化 编译器 而言，main 中g的值看上去从来没有变化过，因此使用缓存在寄存器中g 的副本来满足对g 的每次引用是很安全的。如果这样，main 函数可能永远都无法看到处理程序更新过的值。可以用volatile 类型限定符来定义一个变量，告诉编译器不要缓存这个变量。例如：volatile 限定符强迫编译器每次在代码中引用g时，都要从内存中读取g的值。一般来说，和其他所有共享数据结构一样，应该暂时阻塞信号，保护每次对全局变量的访问。

```
volatile int g;
```

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

6. 用sig_atomic_t声明标志。在常见的处理程序设计中，处理程序会写全局标志来记录收到了信号。主程序周期性地读这个标志，响应信号，再清除该标志。对于通过这种方式来共享的标志，C 提供一种整型数据类型sig_atomic_t对它的读和写保证会是原子的（不可中断的）。

7. 信号的一个与直觉不符的方面是未处理的信号是不排队的。因为 pending 位向量中每种类型的信号只对应有一位，所以每种类型最多只能有一个未处理的信号。关键思想是如果存在一个未处理的信号就表明至少有一个信号到达了。

进程调度的时机

1. 当前运行的进程运行结束。
2. 当前运行的进程由于某种原因阻塞。
3. 执行完系统调用等系统程序后返回用户进程。
4. 在使用抢占调度的系统中，具有更高优先级的进程就绪时。
5. 分时系统中，分给当前进程的时间片用完。

不能进行进程调度的情况

1. 在中断处理程序执行时。
2. 在操作系统的内核程序临界区内。
3. 其它需要完全屏蔽中断的原子操作过程中。

进程的调度策略

1. 先到先服务调度算法
2. 短作业优先调度算法
3. 优先级调度算法
4. 时间片轮转调度算法

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

5. 高响应比优先调度算法
6. 多级队列调度算法
7. 多级反馈队列调度算法

进程调度策略的基本设计指标

1. CPU利用率
2. 系统吞吐率，即单位时间内CPU完成的作业的数量。
3. 响应时间。
4. 周转时间。是指作业从提交到完成的时间间隔。从每个作业的角度看，完成每个作业的时间也是很关键
    - 平均周转时间
    - 带权周转时间
    - 平均带权周转时间

进程的状态与状态转换

进程在运行时有三种基本状态：就绪态、运行态和阻塞态。

1. 运行（running）态：进程占有处理器正在运行的状态。进程已获得CPU，其程序正在执行。在单处理机系统中，只有一个进程处于执行状态； 在多处理机系统中，则有多个进程处于执行状态。

2.就绪（ready）态：进程具备运行条件，等待系统分配处理器以便运行的状态。 当进程已分配到除CPU以外的所有必要资源后，只要再获得CPU，便可立即执行，进程这时的状态称为就绪状态。在一个系统中处于就绪状态的进程可能有多个，通常将它们排成一个队列，称为就绪队列。

3.阻塞（wait）态：又称等待态或睡眠态，指进程不具备运行条件，正在等待某个时间完成的状态。

各状态之间的转换：

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

1. 就绪→执行 处于就绪状态的进程，当进程调度程序为之分配了处理机后，该进程便由就绪状态转变成执行状态。
2. 执行→就绪 处于执行状态的进程在其执行过程中，因分配给它的一个时间片已用完而不得不让出处理机，于是进程从执行状态转变成就绪状态。
3. 执行→阻塞 正在执行的进程因等待某种事件发生而无法继续执行时，便从执行状态变成阻塞状态。
4. 阻塞→就绪 处于阻塞状态的进程，若其等待的事件已经发生，于是进程由阻塞状态转变为就绪状态。

什么是孤儿进程？僵尸进程？

1. 孤儿进程： 父进程退出，子进程还在运行的这些子进程都是孤儿进程，孤儿进程将被init进程（1号进程）所收养，并由init进程对他们完成状态收集工作。

2。 僵尸进程： 进程使用fork创建子进程，如果子进程退出，而父进程并没有调用wait 获waitpid 获取子进程的状态信息，那么子进程的进程描述符仍然保存在系统中的这些进程是僵尸进程。

什么是线程？

1. 是进程划分的任务，是一个进程内可调度的实体，是CPU调度的基本单位，用于保证程序的实时性，实现进程内部的并发。
2. 线程是操作系统可识别的最小执行和调度单位。每个线程都独自占用一个虚拟处理器：独自的寄存器组，指令计数器和处理器状态。
3. 每个线程完成不同的任务，但是属于同一个进程的不同线程之间共享同一地址空间（也就是同样的动态内存，映射文件，目标代码等等），打开的文件队列和其他内核资源。

为什么需要线程？

线程产生的原因：进程可以使多个程序能并发执行，以提高资源的利用率和系统的吞吐量；但是其具有一些缺点：

1. 进程在同一时刻只能做一个任务，很多时候不能充分利用CPU资源。
2. 进程在执行的过程中如果发生阻塞，整个进程就会挂起，即使进程中其它任务不依赖于等待的资源，进程仍会被阻塞。

引入线程就是为了解决以上进程的不足，线程具有以下的优点：

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

1. 从资源上来讲，开辟一个线程所需要的资源要远小于一个进程。

2. 从切换效率上来讲，运行于一个进程中的多个线程，它们之间使用相同的地址空间，而且线程间彼此切换所需时间也远远小于进程间切换所需要的时间（这种时间的差异主要由于缓存的大量未命中导致）。

3. 从通信机制上来讲，线程间方便的通信机制。对不同进程来说，它们具有独立的地址空间，要进行数据的传递只能通过进程间通信的方式进行。线程则不然，属于同一个进程的不同线程之间共享同一地址空间，所以一个线程的数据可以被其它线程感知，线程间可以直接读写进程数据段（如全局变量）来进行通信（需要一些同步措施）。

简述线程和进程的区别和联系

1. 一个线程只能属于一个进程，而一个进程可以有多个线程，但至少有一个线程。线程依赖于进程而存在。

2. 进程在执行过程中拥有独立的地址空间，而多个线程共享进程的地址空间。（资源分配给进程，同一进程的所有线程共享该进程的所有资源。同一进程中的多个线程共享代码段（代码和常量），数据段（全局变量和静态变量），扩展段（堆存储）。但是每个线程拥有自己的栈段，栈段又叫运行时段，用来存放所有局部变量和临时变量。）

3. 进程是资源分配的最小单位，线程是CPU调度的最小单位。

4. 通信：由于同一进程中的多个线程具有相同的地址空间，使它们之间的同步和通信的实现，也变得比较容易。进程间通信 `IPC`，线程间可以直接读写进程数据段（如全局变量）来进行通信（需要一些同步方法，以保证数据的一致性）。

5. 进程编程调试简单可靠性高，但是创建销毁开销大；线程正相反，开销小，切换速度快，但是编程调试相对复杂。

6. 进程间不会相互影响；一个进程内某个线程挂掉将导致整个进程挂掉。

7. 进程适应于多核、多机分布；线程适用于多核。

进程和线程的基本API

进程API以Unix系统为例，线程相关的API属于Posix线程(Pthreads)标准接口。

进程原语	线程原语	描述
`fork`	`pthread_create`	创建新的控制流
`exit`	`pthread_exit`	从现有的控制流中退出
`waitpid`	`pthread_join`	从控制流中得到退出状态

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者昵称：kuangd_1992

进程原语	线程原语	描述
`atexit`	`pthread_cancel_push`	注册在退出控制流时调用的函数
`getpid`	`pthread_self`	获取控制流的ID
`abort`	`pthread_cancel`	请求控制流的非正常退出

多线程模型

1. 多对一模型。将多个用户级线程映射到一个内核级线程上。该模型下，线程在用户空间进行管理，效率较高。缺点就是一个线程阻塞，整个进程内的所有线程都会阻塞。几乎没有系统继续使用这个模型。
2. 一对一模型。将内核线程与用户线程一一对应。优点是一个线程阻塞时，不会影响到其它线程的执行。该模型具有更好的并发性。缺点是内核线程数量一般有上限，会限制用户线程的数量。更多的内核线程数目也给线程切换带来额外的负担。linux和Windows操作系统家族都是使用一对一模型。
3. 多对多模型。将多个用户级线程映射到多个内核级线程上。结合了多对一模型和一对一模型的特点。

进程同步的方法

操作系统中，进程是具有不同的地址空间的，两个进程是不能感知到对方的存在的。有时候，需要多个进程来协同完成一些任务。
当多个进程需要对同一个内核资源进行操作时，这些进程便是竞争的关系，操作系统必须协调各个进程对资源的占用，进程的互斥是解决进程间竞争关系的方法。 进程互斥指若干个进程要使用同一共享资源时，任何时刻最多允许一个进程去使用，其他要使用该资源的进程必须等待，直到占有资源的进程释放该资源。
当多个进程协同完成一些任务时，不同进程的执行进度不一致，这便产生了进程的同步问题。需要操作系统干预，在特定的同步点对所有进程进行同步，这种协作进程之间相互等待对方消息或信号的协调关系称为进程同步。进程互斥本质上也是一种进程同步。
进程的同步方法：

1. 互斥锁
2. 读写锁
3. 条件变量
4. 记录锁(record locking)
5. 信号量

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

6. 屏障（barrier）

线程同步的方法

操作系统中，属于同一进程的线程之间具有相同的地址空间，线程之间共享数据变得简单高效。遇到竞争的线程同时修改同一数据或是协作的线程设置同步点的问题时，需要使用一些线程同步的方法来解决这些问题。

线程同步的方法：

1. 互斥锁
2. 读写锁
3. 条件变量
4. 信号量
5. 自旋锁
6. 屏障（barrier）

进程同步与线程同步有什么区别

进程之间地址空间不同，不能感知对方的存在，同步时需要将锁放在多进程共享的空间。而线程之间共享同一地址空间，同步时把锁放在所属的同一进程空间即可。

死锁是怎样产生的?

死锁是指两个或两个以上进程在执行过程中，因争夺资源而造成的下相互等待的现象。
产生死锁需要满足下面四个条件：

1. 互斥条件：进程对所分配到的资源不允许其他进程访问，若其他进程访问该资源，只能等待，直至占有该资源的进程使用完成后释放该资源。
2. 占有并等待条件：进程获得一定的资源后，又对其他资源发出请求，但是该资源可能被其他进程占有，此时请求阻塞，但该进程不会释放自己已经占有的资源。
3. 非抢占条件：进程已获得的资源，在未完成使用之前，不可被剥夺，只能在使用后自己释放。

版权声明：本文为博主原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接和本声明。
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

4. 循环等待条件：进程发生死锁后，必然存在一个进程-资源之间的环形链。

如何解决死锁问题?

解决死锁的方法即破坏产生死锁的四个必要条件之一，主要方法如下:

1. 资源一次性分配，这样就不会再有请求了（破坏请求条件）。
2. 只要有一个资源得不到分配，也不给这个进程分配其他的资源（破坏占有并等待条件）。
3. 可抢占资源：即当进程新的资源未得到满足时，释放已占有的资源，从而破坏不可抢占的条件。
4. 资源有序分配法：系统给每类资源赋予一个序号，每个进程按编号递增的请求资源，释放则相反，从而破坏环路等待的条件

什么是虚拟地址，什么是物理地址?

地址空间是一个非负整数地址的有序集合。

在一个带虚拟内存的系统中，CPU 从一个有N=pow(2,n)个地址的地址空间中生成虚拟地址，这个地址空间称为虚拟地址空间（virtual address space）,现代系统通常支持 32 位或者 64 位虚拟地址空间。

一个系统还有一个物理地址空间（physical address space），对应于系统中物理内存的M 个字节。

地址空间的概念是很重要的，因为它清楚地区分了数据对象（字节）和它们的属性（地址）。

一旦认识到了这种区别，那么我们就可以将其推广，允许每个数据对象有多个独立的地址，其中每个地址都选自一个不同的地址空间。这就是虚拟内存的基本思想。

主存中的每字节都有一个选自虚拟地址空间的虚拟地址和一个选自物理地址空间的物理地址。

什么是虚拟内存?

为了更加有效地管理内存并且少出错，现代系统提供了一种对主存的抽象概念，叫做虚拟内存(VM)。虚拟内存是硬件异常、硬件地址翻译、主存、磁盘文件和内核软件的完美交互，它为每个进程提供了一个大的、一致的和私有的地址空间。通过一个很清晰的机制，虚拟内存提供了三个重要的能力：

1. 它将主存看成是一个存储在磁盘上的地址空间的高速缓存，在主存中只保存活动区域，并根据需要在磁盘和主存之间来回传送数据，通过这种方式，它高效地使用了主存。

内容来源：csdn.net
作者昵称：kuangd_1992
作者主页：https://blog.csdn.net/kuangd_1992

2. 它为每个进程提供了一致的地址空间，从而简化了内存管理。

3. 它保护了每个进程的地址空间不被其他进程破坏。

为什么要引入虚拟内存?

1. 虚拟内存作为缓存的工具

   - 虚拟内存被组织为一个由存放在磁盘上的N个连续的字节大小的单元组成的数组。
   - 虚拟内存利用DRAM缓存来自通常更大的虚拟地址空间的页面。

2. 虚拟内存作为内存管理的工具。操作系统为每个进程提供了一个独立的页表，也就是独立的虚拟地址空间。多个虚拟页面可以映射到同一个物理页面上。

   - **简化链接：** 独立的地址空间允许每个进程的内存映像使用相同的基本格式，而不管代码和数据实际存放在物理内存的何处。

     - 例如：一个给定的 `linux` 系统上的每个进程都是用类似的内存格式，对于64为地址空间，代码段总是从虚拟地址） `0x400000` 开始，数据段，代码段，栈，堆等等。

   - **简化加载：** 虚拟内存还使得容易向内存中加载可执行文件和共享对象文件。要把目标文件中.text和.data节加载到一个新创建的进程中，Linux加载器为代码和数据段分配虚拟页VP，把他们**标记为无效（未被缓存）**，将页表条目指向目标文件的起始位置。

     - **加载器从不在磁盘到内存实际复制任何数据，在每个页初次被引用时，虚拟内存系统会按照需要自动的调入数据页。**

   - **简化共享：** 独立地址空间为OS提供了一个管理用户进程和操作系统自身之间共享的一致机制。

     - 一般：每个进程有各自私有的代码，数据，堆栈，是不和其他进程共享的，**这样OS创建页表，将虚拟页映射到不连续的物理页面。**
     - 某些情况下，需要进程来共享代码和数据。例如每个进程调用相同的操作系统内核代码，或者C 标准库函数。**OS会把不同进程中适当的虚拟页面映射到相同的物理页面。**

   - **简化内存分配：** 虚拟内存向用户提供一个简单的分配额外内存的机制。当一个运行在用户进程中的程序要求额外的堆空间时（如 `malloc` ），OS分配一个适当k大小个连续的虚拟内存页面，并且将他们映射到物理内存中任意位置的k个任意物理页面，**因此操作系统没有必要分配k个连续的物理内存页面，页面可以随机的分散在物理内存中。**

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

- 虚拟内存作为内存保护的工具。不应该允许一个用户进程修改它的只读段，也不允许它修改任何内核代码和数据结构，不允许读写其他进程的私有内存，不允许修改任何与其他进程共享的虚拟页面。每次CPU生成一个地址时，`MMU`会读一个`PTE`，通过在`PTE`上添加一些额外的许可位来控制对一个虚拟页面内容的访问十分简单。

常见的页面置换算法

当访问一个内存中不存在的页，并且内存已满，则需要从内存中调出一个页或将数据送至磁盘对换区，替换一个页，这种现象叫做缺页置换。当前操作系统最常采用的缺页置换算法如下：

- 先进先出(FIFO)算法：
  - 思路：置换最先调入内存的页面，即置换在内存中驻留时间最久的页面。
  - 实现：按照进入内存的先后次序排列成队列，从队尾进入，从队首删除。
  - 特点：实现简单；性能较差，调出的页面可能是经常访问的
- 最近最少使用（`LRU`）算法:
  - 思路： 置换最近一段时间以来最长时间未访问过的页面。根据程序局部性原理，刚被访问的页面，可能马上又要被访问；而较长时间内没有被访问的页面，可能最近不会被访问。
  - 实现：缺页时，计算内存中每个逻辑页面的上一次访问时间，选择上一次使用到当前时间最长的页面
  - 特点：可能达到最优的效果，维护这样的访问链表开销比较大

当前最常采用的就是`LRU`算法。

- 最不常用算法（`Least Frequently Used, LFU`）
  - 思路：缺页时，置换访问次数最少的页面
  - 实现：每个页面设置一个访问计数，访问页面时，访问计数加1，缺页时，置换计数最小的页面
  - 特点：算法开销大，开始时频繁使用，但以后不使用的页面很难置换

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

请说一下什么是写时复制?

- 如果有多个进程要读取它们自己的那部门资源的副本，那么复制是不必要的。每个进程只要保存一个指向这个资源的指针就可以了。只要没有进程要去修改自己的“副本”，就存在着这样的幻觉：每个进程好像独占那个资源。从而就避免了复制带来的负担。如果一个进程要修改自己的那份资源“副本”，那么就会复制那份资源，并把复制的那份提供给进程。不过其中的复制对进程来说是透明的。这个进程就可以修改复制后的资源了，同时其他的进程仍然共享那份没有修改过的资源。所以这就是名称的由来：在写入时进行复制。
- 写时复制的主要好处在于：如果进程从来就不需要修改资源，则不需要进行复制。惰性算法的好处就在于它们尽量推迟代价高昂的操作，直到必要的时刻才会去执行。
- 在使用虚拟内存的情况下，写时复制（Copy-On-Write）是以页为基础进行的。所以，只要进程不修改它全部的地址空间，那么就不必复制整个地址空间。在fork()调用结束后，父进程和子进程都相信它们有一个自己的地址空间，但实际上它们共享父进程的原始页，接下来这些页又可以被其他的父进程或子进程共享。

实时操作系统的概念

实时操作系统（Real-time operating system, RTOS），又称即时操作系统，它会按照排序运行、管理系统资源，并为开发应用程序提供一致的基础。 实时操作系统与一般的操作系统相比，最大的特色就是“实时性”，如果有一个任务需要执行，实时操作系统会马上（在较短时间内）执行该任务，不会有较长的延时。这种特性保证了各个任务的及时执行。

优先级反转是什么？如何解决

由于多进程共享资源，具有最高优先权的进程被低优先级进程阻塞，反而使具有中优先级的进程先于高优先级的进程执行，导致系统的崩溃。这就是所谓的优先级反转(Priority Inversion)。其实,优先级反转是在高优级(假设为A)的任务要访问一个被低优先级任务(假设为C)占有的资源时,被阻塞.而此时又有优先级高于占有资源的任务©而低于被阻塞的任务(A)的优先级的任务(假设为B)时,于是,占有资源的任务就被挂起(占有的资源仍为它占有),因为占有资源的任务优先级很低,所以,它可能一直被另外的任务挂起.而它占有的资源也就一直不能释放,这样,引起任务A一直没办法执行.而比它优先低的任务却可以执行。

目前解决优先级反转有许多种方法。其中普遍使用的有2种方法：一种被称作优先级继承(priority inheritance)；另一种被称作优先级极限(priority ceilings)。

1. 优先级继承(priority inheritance) 优先级继承是指将低优先级任务的优先级提升到等待它所占有的资源的最高优先级任务的优先级.当高优先级任务由于等待资源而被阻塞时,此时资源的拥有者的优先级将会自动被提升。
2. 优先级天花板(priority ceilings)优先级天花板是指将申请某资源的任务的优先级提升到可能访问该资源的所有任务中最高优先级任务的优先级.(这个优先级称为该资源的优先级天花板)。

版权声明：本文为博主原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接和本声明。
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

操作系统面试题二

说下进程的状态

> 原文：https://zwmst.com/1318.html

就绪：进程已处于准备好运行的状态，即进程已分配到除CPU外的所有必要资源后，只要再获 得CPU，便可立即执行

执行：进程已经获得CPU，程序正在执行状态

阻塞：正在执行的进程由于发生某事件（如I/O请求、申请缓冲区失败等）暂时无法继续执行 的状态

说下进程和线程的联系与区别

> 原文：https://zwmst.com/1320.html

进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动，进程是系统进行资源分 配和调度的一个独立单位

线程是进程的一个实体，是CPU调度和分派的基本单位，它是比进程更小的能独立运行的基本 单位

**进程和线程的关系

一个线程只能属于一个进程，而一个进程可以有多个线程，但至少有一个线程。线程是操作系 统可识别的最小执行和调度单位

资源分配给进程，同一进程的所有线程共享该进程的所有资源。 同一进程中的多个线程共享代 码段(代码和常量)，数据段(全局变量和静态变量)，扩展段(堆存储)。但是每个线程拥有自己的栈段，栈段又叫运行时段，用来存放所有局部变量和临时变量

处理机分给线程，即真正在处理机上运行的是线程

线程在执行过程中，需要协作同步。不同进程的线程间要利用消息通信的办法实现同步

**进程与线程的区别

进程有自己的独立地址空间，线程没有

进程是资源分配的最小单位，线程是CPU调度的最小单位

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

进程和线程通信方式不同(线程之间的通信比较方便。同一进程下的线程共享数据（比如全局变 量，静态变量），通过这些数据来通信不仅快捷而且方便，当然如何处理好这些访问的同步与 互斥正是编写多线程程序的难点。而进程之间的通信只能通过进程通信的方式进行。)

进程上下文切换开销大，线程开销小

一个进程挂掉了不会影响其他进程，而线程挂掉了会影响其他线程

对进程进程操作一般开销都比较大，对线程开销就小了

为什么进程上下文切换比线程上下文切换代价高?

> 原文：https://zwmst.com/1322.html

**进程切换分两步：

切换页目录以使用新的地址空间

切换内核栈和硬件上下文

对于linux来说，线程和进程的最大区别就在于地址空间，对于线程切换，第1步是不需要做 的，第2是进程和线程切换都要做的

**切换的性能消耗：

线程上下文切换和进程上下问切换一个最主要的区别是线程的切换虚拟内存空间依然是相同 的，但是进程切换是不同的。这两种上下文切换的处理都是通过操作系统内核来完成的。内核 的这种切换过程伴随的最显著的性能损耗是将寄存器中的内容切换出

另外一个隐藏的损耗是上下文的切换会扰乱处理器的缓存机制。简单的说，一旦去切换上下 文，处理器中所有已经缓存的内存地址一瞬间都作废了。还有一个显著的区别是当你改变虚拟 内存空间的时候，处理的页表缓冲（processor’s Translation Lookaside Buffer (TLB)）或 者相当的神马东西会被全部刷新，这将导致内存的访问在一段时间内相当的低效。但是在线程 的切换中，不会出现这个问题

说下你对进程同步的理解

> 原文：https://zwmst.com/1324.html

进程同步的主要任务：是对多个相关进程在执行次序上进行协调，以使并发执行的诸进程之间 能有效地共享资源和相互合作，从而使程序的执行具有可再现性

内容来源：csdn.net
作者昵称：kuangd_1992
作者主页：https://blog.csdn.net/kuangd_1992

同步机制遵循的原则：

空闲让进；

忙则等待（保证对临界区的互斥访问）；

有限等待（有限代表有限的时间，避免死等）；

让权等待（当进程不能进入自己的临界区时，应该释放处理机，以免陷入忙等状态）

进程的通信方式有哪些

> 原文：https://zwmst.com/1326.html

进程通信，是指进程之间的信息交换（信息量少则一个状态或数值，多者则是成千上万个字 节）。因此，对于用信号量进行的进程间的互斥和同步，由于其所交换的信息量少而被归结为 低级通信

所谓高级进程通信指：用户可以利用操作系统所提供的一组通信命令传送大量数据的一种通信 方式。操作系统隐藏了进程通信的实现细节。或者说，通信过程对用户是透明的

高级通信机制可归结为三大类：

共享存储器系统（存储器中划分的共享存储区）；实际操作中对应的是“剪贴板”（剪贴板实际 上是系统维护管理的一块内存区域）的通信方式，比如举例如下：word进程按下ctrl+c，在ppt进程按下ctrl+v，即完成了word进程和ppt进程之间的通信，复制时将数据放入到剪贴 板，粘贴时从剪贴板中取出数据，然后显示在ppt窗口上

消息传递系统（进程间的数据交换以消息（message）为单位，当今最流行的微内核操作系统 中，微内核与服务器之间的通信，无一例外地都采用了消息传递机制。应用举例：邮槽

（MailSlot）是基于广播通信体系设计出来的，它采用无连接的不可靠的数据传输。邮槽是一 种单向通信机制，创建邮槽的服务器进程读取数据，打开邮槽的客户机进程写入数据

管道通信系统（管道即：连接读写进程以实现他们之间通信的共享文件（pipe文件，类似先进 先出的队列，由一个进程写，另一进程读））。实际操作中，管道分为：匿名管道、命名管 道。匿名管道是一个未命名的、单向管道，通过父进程和一个子进程之间传输数据。匿名管道 只能实现本地机器上两个进程之间的通信，而不能实现跨网络的通信。命名管道不仅可以在本 机上实现两个进程间的通信，还可以跨网络实现两个进程间的通信

版权声明：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

管道：管道是单向的、先进先出的、无结构的、固定大小的字节流，它把一个进程的标准输出 和另一个进程的标准输入连接在一起。写进程在管道的尾端写入数据，读进程在管道的道端读 出数据。数据读出后将从管道中移走，其它读进程都不能再读到这些数据。管道提供了简单的 流控制机制。进程试图读空管道时，在有数据写入管道前，进程将一直阻塞。同样地，管道已 经满时，进程再试图写管道，在其它进程从管道中移走数据之前，写进程将一直阻塞。

信号量：信号量是一个计数器，可以用来控制多个进程对共享资源的访问。它常作为一种锁机 制，防止某进程正在访问共享资源时，其它进程也访问该资源。因此，主要作为进程间以及同 一进程内不同线程之间的同步手段

消息队列：是一个在系统内核中用来保存消 息的队列，它在系统内核中是以消息链表的形式出 现的。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺 点

共享内存：共享内存允许两个或多个进程访问同一个逻辑内存。这一段内存可以被两个或两个 以上的进程映射至自身的地址空间中，一个进程写入共享内存的信息，可以被其他使用这个共 享内存的进程，通过一个简单的内存读取读出，从而实现了进程间的通信。如果某个进程向共 享内存写入数据，所做的改动将立即影响到可以访问同一段共享内存的任何其他进程。共享内 存是最快的IPC方式，它是针对其它进程间通信方式运行效率低而专门设计的。它往往与其它 通信机制（如信号量）配合使用，来实现进程间的同步和通信

套接字：套接字也是一种进程间通信机制，与其它通信机制不同的是，它可用于不同机器间的 进程通信

进程调度的种类有哪些?

> 原文：https://zwmst.com/1328.html

高级调度：(High-Level Scheduling)又称为作业调度，它决定把后备作业调入内存运行

低级调度：(Low-Level Scheduling)又称为进程调度，它决定把就绪队列的某进程获得CPU

中级调度：(Intermediate-Level Scheduling)又称为在虚拟存储器中引入，在内、外存对换 区进行进程对换

非抢占式调度与抢占式调度的区别是什么?

> 原文：https://zwmst.com/1330.html

非抢占式：分派程序一旦把处理机分配给某进程后便让它一直运行下去，直到进程完成或发生 进程调度进程调度某事件而阻塞时，才把处理机分配给另一个进程

抢占式：操作系统将正在运行的进程强行暂停，由调度程序将CPU分配给其他就绪进程的调度 方式

文章来自：博客园
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

说下你知道的调度算法

> 原文：https://zwmst.com/1332.html

FIFO或First Come, First Served (FCFS)先来先服务

调度的顺序就是任务到达就绪队列的顺序

公平、简单(FIFO队列)、非抢占、不适合交互式

未考虑任务特性，平均等待时间可以缩短

Shortest Job First (SJF)

最短的作业(CPU区间长度最小)最先调度

SJF可以保证最小的平均等待时间

Shortest Remaining Job First (SRJF)

SJF的可抢占版本，比SJF更有优势

SJF(SRJF): 如何知道下一CPU区间大小？根据历史进行预测: 指数平均法

优先权调度

每个任务关联一个优先权，调度优先权最高的任务

注意：优先权太低的任务一直就绪，得不到运行，出现“饥饿”现象

Round-Robin(RR)轮转调度算法

设置一个时间片，按时间片来轮转调度（“轮叫”算法）

优点: 定时有响应，等待时间较短；缺点: 上下文切换次数较多

时间片太大，响应时间太长；吞吐量变小，周转时间变长；当时间片过长时，退化为FCFS

多级队列调度

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

按照一定的规则建立多个进程队列

不同的队列有固定的优先级（高优先级有抢占权）

不同的队列可以给不同的时间片和采用不同的调度方法

存在问题1：没法区分I/O bound和CPU bound

存在问题2：也存在一定程度的“饥饿”现象

多级反馈队列

在多级队列的基础上，任务可以在队列之间移动，更细致的区分任务

可以根据“享用”CPU时间多少来移动队列，阻止“饥饿”

最通用的调度算法，多数OS都使用该方法或其变形，如UNIX、Windows等

一个程序从开始运行到结束的完整过程（四个过程）

> 原文：https://zwmst.com/1334.html

预处理：条件编译，头文件包含，宏替换的处理，生成.i文件。

编译：将预处理后的文件转换成汇编语言，生成.s文件

汇编：汇编变为目标代码(机器代码)生成.o的文件

链接：连接目标代码,生成可执行程序

死锁出现的条件?

> 原文：https://zwmst.com/1336.html

定义:如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件,那么 该组进程就是死锁的。或者在两个或多个并发进程中，如果每个进程持有某种资源而又都等待 别的进程释放它或它们现在保持着的资源，在未改变这种状态之前都不能向前推进，称这一组 进程产生了死锁。通俗地讲，就是两

内容来源：csdn.net
作者主页：https://blog.csdn.net/kuangd_1992

个或多个进程被无限期地阻塞、相互等待的一种状态。

**产生死锁的必要条件：

互斥条件(Mutual exclusion)：资源不能被共享，只能由一个进程使用。

请求与保持条件(Hold and wait)：已经得到资源的进程可以再次申请新的资源。

非抢占条件(No pre-emption)：已经分配的资源不能从相应的进程中被强制地剥夺。

循环等待条件(Circular wait)：系统中若干进程组成环路，该环路中每个进程都在等待相邻进 程正占用的资源。

如何处理死锁问题

> 原文：https://zwmst.com/1338.html

忽略该问题。例如鸵鸟算法，该算法可以应用在极少发生死锁的的情况下。为什么叫鸵鸟算法 呢，因为传说中鸵鸟看到危险就把头埋在地底下，可能鸵鸟觉得看不到危险也就没危险了吧。 跟掩耳盗铃有点像。

检测死锁并且恢复。

仔细地对资源进行动态分配，使系统始终处于安全状态以避免死锁。

通过破除死锁四个必要条件之一，来防止死锁产生。

什么是临界资源

> 原文：https://zwmst.com/1341.html

在操作系统中，进程是占有资源的最小单位（线程可以访问其所在进程内的所有资源，但线程 本身并不占有资源或仅仅占有一点必须资源）。但对于某些资源来说，其在同一时间只能被一 个进程所占用。这些一次只能被一个进程所占用的资源就是所谓的临界资源。典型的临界资源 比如物理上的打印机，或是存在硬盘或内存中被多个进程所共享的一些变量和数据等(如果这类 资源不被看成临界资源加以保护，那么很有可能造成丢数据的问题)。

对于临界资源的访问，必须是互斥进行。也就是当临界资源被占用时，另一个申请临界资源的 进程会被阻塞，直到其所申请的临界资源被释放。而进程内访问临界资源的代码被成为临界 区。

内容来源：csdn.net
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

介绍一下内存池、进程池、线程池

> 原文：https://zwmst.com/1343.html

首先介绍一个概念“池化技术 ”。池化技术就是：提前保存大量的资源，以备不时之需以及重 复使用。池化技术应用广泛，如内存池，线程池，连接池等等。内存池相关的内容，建议看看 Apache 、Nginx等开源web服务器的内存池实现。

由于在实际应用当做，分配内存、创建进程、线程都会设计到一些系统调用，系统调用需要导 致程序从用户态切换到内核态，是非常耗时的操作。因此，当程序中需要频繁的进行内存申请 释放，进程、线程创建销毁等操作时，通常会使用内存池、进程池、线程池技术来提升程序的 性能。

线程池：线程池的原理很简单，类似于操作系统中的缓冲区的概念，它的流程如下：先启动若 干数量的线程，并让这些线程都处于睡眠状态，当需要一个开辟一个线程去做具体的工作时， 就会唤醒线程池中的某一个睡眠线程，让它去做具体工作，当工作完成后，线程又处于睡眠状 态，而不是将线程销毁。

进程池与线程池同理。

内存池：内存池是指程序预先从操作系统申请一块足够大内存，此后，当程序中需要申请内存 的时候，不是直接向操作系统申请，而是直接从内存池中获取；同理，当程序释放内存的时 候，并不真正将内存返回给操作系统，而是返回内存池。当程序退出(或者特定时间)时，内存 池才将之前申请的内存真正释放。

动态链接库与静态链接库的区别

> 原文：https://zwmst.com/1345.html

静态库

静态库是一个外部函数与变量的集合体。静态库的文件内容，通常包含一堆程序员自定的变量 与函数，其内容不像动态链接库那么复杂，在编译期间由编译器与链接器将它集成至应用程序 内，并制作成目标文件以及可以独立运作的可执行文件。而这个可执行文件与编译可执行文件 的程序，都是一种程序的静态创建（static build）。

动态库

静态库很方便，但是如果我们只是想用库中的某一个函数，却仍然得把所有的内容都链接进 去。一个更现代的方法则是使用共享库，避免了在文件中静态库的大量重复。 动态链接可以在首次载入的时候执行(load-time linking)，这是 Linux 的标准做法，会由动 态链接器ld-linux.so 完成，比方标准 C 库(libc.so) 通常就是动态链接的，这样所有的程序可 以共享同一个库，而不用分别进行封装。 动态链接也可以在程序开始执行的时候完成(run-time linking)，在 Linux 中使用 dlopen() 接口来完成（会使用函数指针），通常用于分布式软件，高性能服务器上。而且共享库也可以 在多个进程间共享。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

链接使得我们可以用多个对象文件构造我们的程序。可以在程序的不同阶段进行（编译、载 入、运行期间均可），理解链接可以帮助我们避免遇到奇怪的错误。

区别：

使用静态库的时候，静态链接库要参与编译，在生成执行文件之前的链接过程中，要将静态链 接库的全部指令直接链接入可执行文件中。而动态库提供了一种方法，使进程可以调用不属于 其可执行代码的函数。函数的可执行代码位于一个.dll文件中，该dll包含一个或多个已被编 译，链接并与使用它们的进程分开储存的函数。 静态库中不能再包含其他动态库或静态库，而在动态库中还可以再包含其他动态或者静态库。 静态库在编译的时候，就将库函数装在到程序中去了，而动态库函数必须在运行的时候才被装 载，所以使用静态库速度快一些。

说下对虚拟内存的理解

> 原文：https://zwmst.com/1347.html

定义：具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充得一种存储器系统。其 逻辑容量由内存之和和外存之和决定。

与传统存储器比较虚拟存储器有以下三个主要特征： 多次性，是指无需在作业运行时一次性地全部装入内存，而是允许被分成多次调入内存运行。 对换性，是指无需在作业运行时一直常驻内存，而是允许在作业的运行过程中，进行换进和换出。 虚拟性，是指从逻辑上扩充内存的容量，使用户所看到的内存容量，远大于实际的内存容量。

虚拟内存的实现有以下两种方式：

请求分页存储管理。

请求分段存储管理。

页面置换算法了解多少?

> 原文：https://zwmst.com/1349.html

操作系统将内存按照页面进行管理，在需要的时候才把进程相应的部分调入内存。当产生缺页 中断时，需要选择一个页面写入。如果要换出的页面在内存中被修改过，变成了“脏”页面，那 就需要先写会到磁盘。页面置换算法，就是要选出最合适的一个页面，使得置换的效率最高。 页面置换算法有很多，简单介绍几个，重点介绍比较重要的LRU及其实现算法。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

一、最优页面置换算法

最理想的状态下，我们给页面做个标记，挑选一个最远才会被再次用到的页面调出。当然，这 样的算法不可能实现，因为不确定一个页面在何时会被用到。

二、先进先出页面置换算法（FIFO）及其改进

这种算法的思想和队列是一样的，该算法总是淘汰最先进入内存的页面，即选择在内存中驻留 时间最久的页面予淘汰。实现：把一个进程已调入内存的页面按先后次序链接成一个队列，并 且设置一个指针总是指向最老的页面。缺点：对于有些经常被访问的页面如含有全局变量、常 用函数、例程等的页面，不能保证这些不被淘汰。

三、最近最少使用页面置换算法LRU（Least Recently Used）

根据页面调入内存后的使用情况做出决策。LRU置换算法是选择最近最久未使用的页面进行淘 汰。

1.为每个在内存中的页面配置一个移位寄存器。(P165）定时信号将每隔一段时间将寄存器右 移一位。最小数值的寄存器对应页面就是最久未使用页面。

2.利用一个特殊的栈保存当前使用的各个页面的页面号。每当进程访问某页面时，便将该页面 的页面号从栈中移出，将它压入栈顶。因此，栈顶永远是最新被访问的页面号，栈底是最近最 久未被访问的页面号。

中断与系统调用了解吗?

> 原文：https://zwmst.com/1351.html

所谓的中断就是在计算机执行程序的过程中，由于出现了某些特殊事情，使得CPU暂停对程序 的执行，转而去执行处理这一事件的程序。等这些特殊事情处理完之后再回去执行之前的程 序。中断一般分为三类：

由计算机硬件异常或故障引起的中断，称为内部异常中断；

由程序中执行了引起中断的指令而造成的中断，称为软中断（这也是和我们将要说明的系统调 用相关的中断)；

由外部设备请求引起的中断，称为外部中断。简单来说，对中断的理解就是对一些特殊事情的 处理。

与中断紧密相连的一个概念就是中断处理程序了。当中断发生的时候，系统需要去对中断进行 处理，对这些中断的处理是由操作系统内核中的特定函数进行的，这些处理中断的特定的函数 就是我们所说的中断处理程序了。

另一个与中断紧密相连的概念就是中断的优先级。中断的优先级说明的是当一个中断正在被处 理的时候，处理器能接受的中断的级别。中断的优先级也表明了中断需要被处理的紧急程度。 每个中断都有一个对应的优先级，当处理器在处理某一中断的时候，只有比这个中断优先级高 的中断可以被处理器接受并且被处理。优先级比这个当前正在被处理的中断优先级要低的中断 将会被忽略。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

典型的中断优先级如下所示： 机器错误 > 时钟 > 磁盘 > 网络设备 > 终端 > 软件中断

在讲系统调用之前，先说下进程的执行在系统上的两个级别：用户级和核心级，也称为用户态 和系统态(user mode and kernel mode)。 用户空间就是用户进程所在的内存区域，相对的，系统空间就是操作系统占据的内存区域。用 户进程和系统进程的所有数据都在内存中。处于用户态的程序只能访问用户空间，而处于内核 态的程序可以访问用户空间和内核空间。

用户态切换到内核态的方式有哪些?

> 原文：https://zwmst.com/1353.html

系统调用：程序的执行一般是在用户态下执行的，但当程序需要使用操作系统提供的服务时， 比如说打开某一设备、创建文件、读写文件（这些均属于系统调用）等，就需要向操作系统发 出调用服务的请求，这就是系统调用。

异常：当CPU在执行运行在用户态下的程序时，发生了某些事先不可知的异常，这时会触发由 当前运行进程切换到处理此异常的内核相关程序中，也就转到了内核态，比如缺页异常。

外围设备的中断：当外围设备完成用户请求的操作后，会向CPU发出相应的中断信号，这时 CPU会暂停执行下一条即将要执行的指令转而去执行与中断信号对应的处理程序，如果先前执 行的指令是用户态下的程序，那么这个转换的过程自然也就发生了由用户态到内核态的切换。 比如硬盘读写操作完成，系统会切换到硬盘读写的中断处理程序中执行后续操作等。

用户态和核心态(内核态）之间的区别是什么呢?

> 原文：https://zwmst.com/1355.html

权限不一样。

用户态的进程能存取它们自己的指令和数据，但不能存取内核指令和数据（或其他进程的指令 和数据）。

核心态下的进程能够存取内核和用户地址某些机器指令是特权指令，在用户态下执行特权指令 会引起错误。在系统中内核并不是作为一个与用户进程平行的估计的进程的集合。

内部碎片与外部碎片分别是什么?

> 原文：https://zwmst.com/1357.html

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

在内存管理中，内部碎片是已经被分配出去的的内存空间大于请求所需的内存空间。

外部碎片是指还没有分配出去，但是由于大小太小而无法分配给申请空间的新进程的内存空间 空闲块。

固定分区存在内部碎片，可变式分区分配会存在外部碎片；

页式虚拟存储系统存在内部碎片；段式虚拟存储系统，存在外部碎片

为了有效的利用内存，使内存产生更少的碎片，要对内存分页，内存以页为单位来使用，最后 一页往往装不满，于是形成了内部碎片。

为了共享要分段，在段的换入换出时形成外部碎片，比如5K的段换出后，有一个4k的段进来 放到原来5k的地方，于是形成1k的外部碎片。

系统调用与库函数的区别

> 原文：https://zwmst.com/1359.html

系统调用(System call)是程序向系统内核请求服务的方式。可以包括硬件相关的服务(例如， 访问硬盘等)，或者创建新进程，调度其他进程等。系统调用是程序和操作系统之间的重要接 口。

库函数：把一些常用的函数编写完放到一个文件里，编写应用程序时调用，这是由第三方提供 的，发生在用户地址空间。

在移植性方面，不同操作系统的系统调用一般是不同的，移植性差；而在所有的ANSI C编译器 版本中，C库函数是相同的。

在调用开销方面，系统调用需要在用户空间和内核环境间切换，开销较大；而库函数调用属于 “过程调用”，开销较小。

守护、僵尸、孤儿进程的概念

> 原文：https://zwmst.com/1361.html

守护进程：运行在后台的一种特殊进程，独立于控制终端并周期性地执行某些任务。

僵尸进程：一个进程 fork 子进程，子进程退出，而父进程没有wait/waitpid子进程，那么子 进程的进程描述符仍保存在系统中，这样的进程称为僵尸进程。

孤儿进程：一个父进程退出，而它的一个或多个子进程还在运行，这些子进程称为孤儿进程。 （孤儿进程将由 init 进程收养并对它们完成状态收集工作）

内容来源：博客
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

操作系统面试题三

一 操作系统基础

面试官顶着蓬松的假发向我走来，只见他一手拿着厚重的 Thinkpad ，一手提着他那淡黄的长裙。

1.1 什么是操作系统?

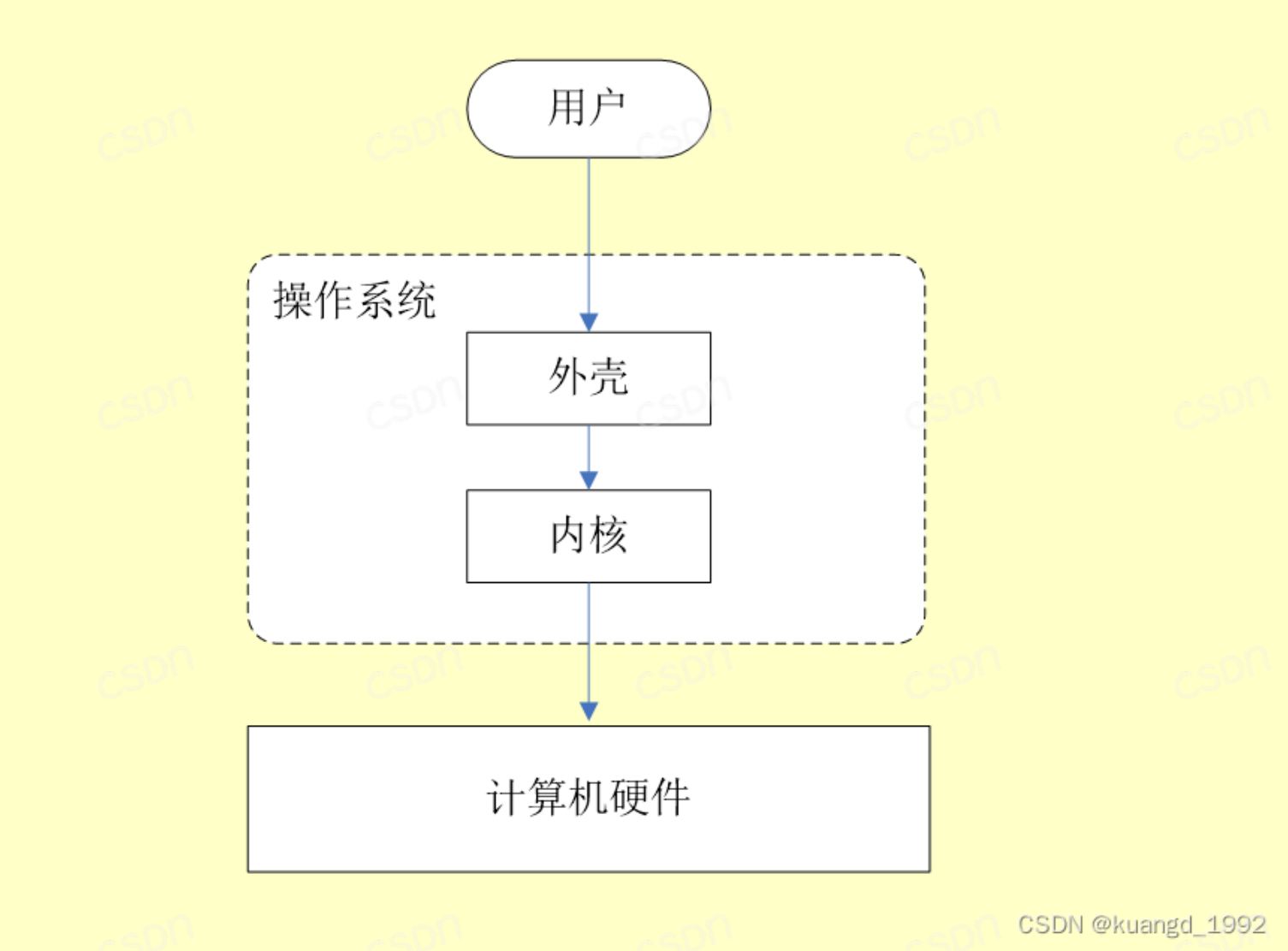
👨‍💼 **面试官**： 先来个简单问题吧！**什么是操作系统?**

🙋‍♂️ **我**： 我通过以下四点向您介绍一下什么是操作系统吧！

1. **操作系统（Operating System，简称 OS）是管理计算机硬件与软件资源的程序，是计算机系统的内核与基石；**
2. **操作系统本质上是运行在计算机上的软件程序；**
3. **操作系统为用户提供一个与系统交互的操作界面；**
4. **操作系统分内核与外壳（我们可以把外壳理解成围绕着内核的应用程序，而内核就是能操作硬件的程序）。**

> 关于内核多插一嘴：内核负责管理系统的进程、内存、设备驱动程序、文件和网络系统等等，决定着系统的性能和稳定性。是连接应用程序和硬件的桥梁。内核就是操作系统背后黑盒的核心。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992



1.2 系统调用

👨‍💼 **面试官：什么是系统调用呢？** 能不能详细介绍一下。

🙋‍♂️ **我：** 介绍系统调用之前，我们先来了解一下用户态和系统态。

根据进程访问资源的特点，我们可以把进程在系统上的运行分为两个级别：

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

1. 用户态(user mode) : 用户态运行的进程或可以直接读取用户程序的数据。
2. 系统态(kernel mode):可以简单的理解系统态运行的进程或程序几乎可以访问计算机的任何资源，不受限制。

说了用户态和系统态之后，那么什么是系统调用呢?

我们运行的程序基本都是运行在用户态，如果我们调用操作系统提供的系统态级别的子功能咋办呢？那就需要系统调用了！

也就是说在我们运行的用户程序中，凡是与系统态级别的资源有关的操作（如文件管理、进程控制、内存管理等)，都必须通过系统调用方式向操作系统提出服务请求，并由操作系统代为完成。

这些系统调用按功能大致可分为如下几类：

- 设备管理。完成设备的请求或释放，以及设备启动等功能。
- 文件管理。完成文件的读、写、创建及删除等功能。
- 进程控制。完成进程的创建、撤销、阻塞及唤醒等功能。
- 进程通信。完成进程之间的消息传递或信号传递等功能。
- 内存管理。完成内存的分配、回收以及获取作业占用内存区大小及地址等功能。

二 进程和线程

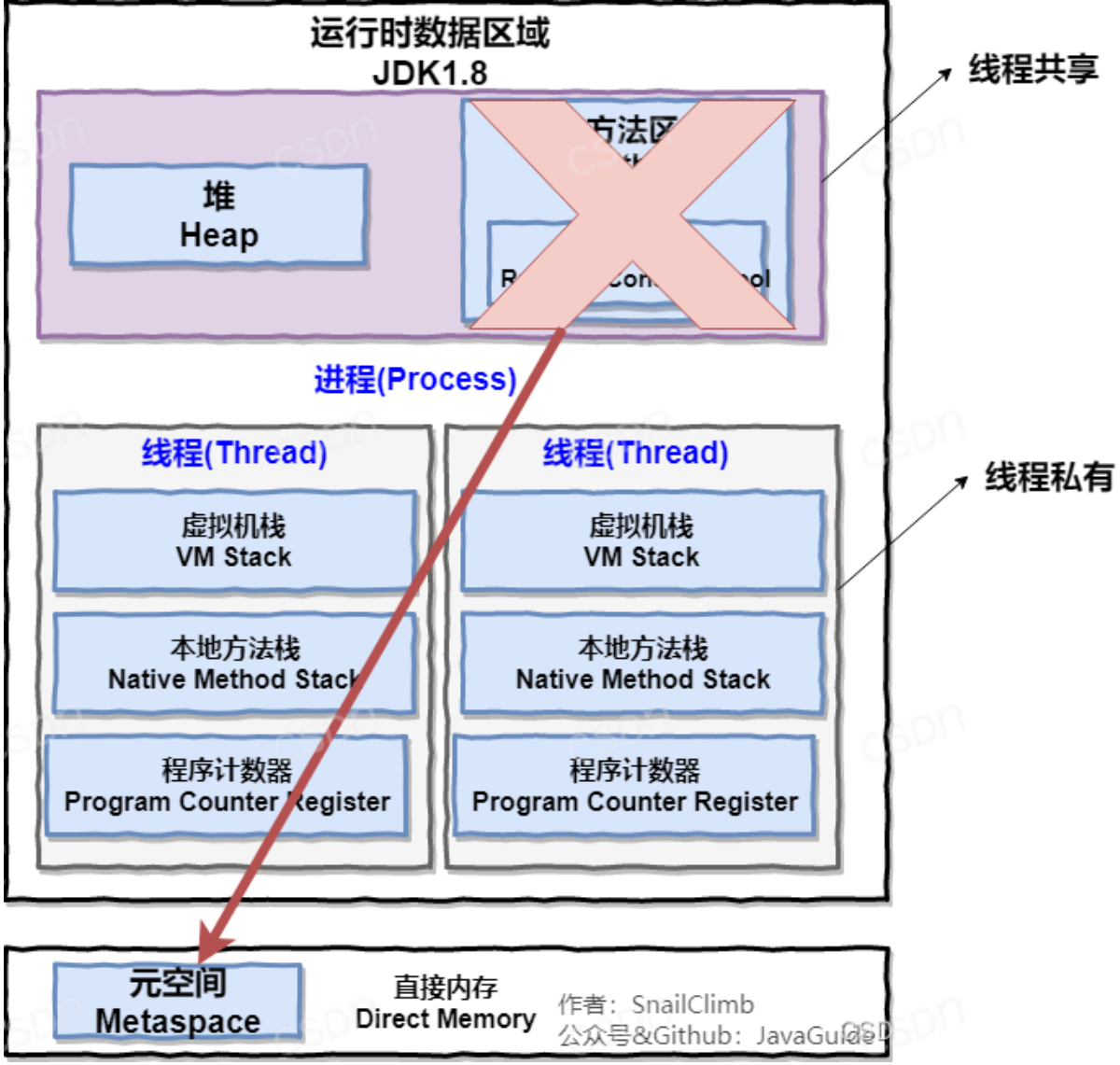
2.1 进程和线程的区别

👮‍♀️ **面试官**: 好的！我明白了！那你再说一下： **进程和线程的区别**。

🙋‍♂️ **我：** 好的！ 下图是 Java 内存区域，我们从 JVM 的角度来说一下线程和进程之间的关系吧！

> 如果你对 Java 内存区域 (运行时数据区) 这部分知识不太了解的话可以阅读一下这篇文章：《可能是把 Java 内存区域讲的最清楚的一篇文章》

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992



从上图可以看出：一个进程中可以有多个线程，多个线程共享进程的**堆**和**方法区 (JDK1.8 之后的元空间)资源，但是每个线程有自己的程序计数器、虚拟机栈** 和**本地方法栈**。

总结： 线程是进程划分成的更小的运行单位,一个进程在其执行的过程中可以产生多个线程。线程和进程最大的不同在于基本上各进程是独立的，而各线程则不一定，因为同一进程中的线程极有可能会相互影响。线程执行开销小，但不利于资源的管理和保护；而进程正相反。

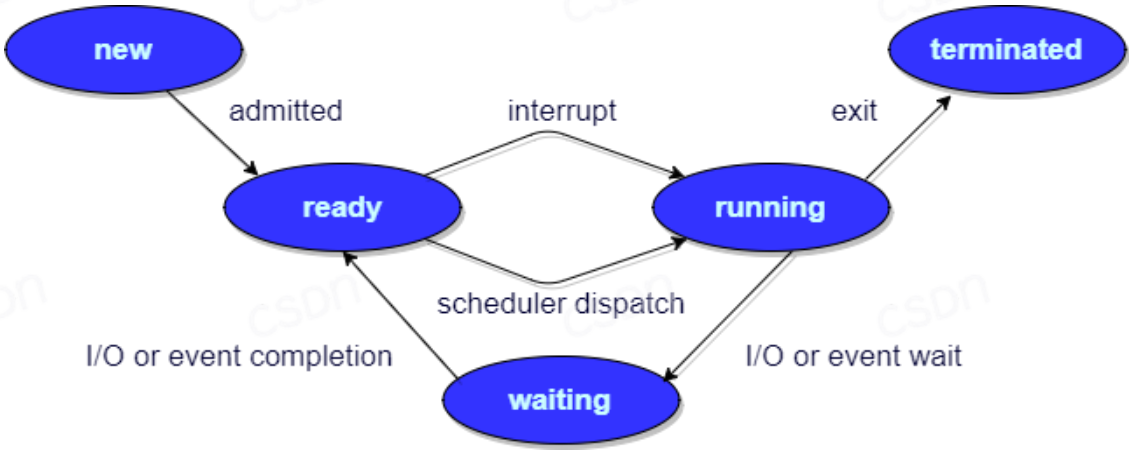
文章来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

2.2 进程有哪几种状态？

👮‍♂️ **面试官：** 那你再说说**进程有哪几种状态?**

🙋‍♂️ **我：** 我们一般把进程大致分为 5 种状态，这一点和线程很像！

- **创建状态(new)**：进程正在被创建，尚未到就绪状态。
- **就绪状态(ready)**：进程已处于准备运行状态，即进程获得了除了处理器之外的一切所需资源，一旦得到处理器资源(处理器分配的时间片)即可运行。
- **运行状态(running)**：进程正在处理器上上运行(单核 CPU 下任意时刻只有一个进程处于运行状态)。
- **阻塞状态(waiting)**：又称为等待状态，进程正在等待某一事件而暂停运行如等待某资源为可用或等待 IO 操作完成。即使处理器空闲，该进程也不能运行。
- **结束状态(terminated)**：进程正在从系统中消失。可能是进程正常结束或其他原因中断退出运行。



2.3 进程间的通信方式

👮‍♂️ **面试官：** **进程间的通信常见的的有哪几种方式呢?**

🙋‍♂️ **我：** 大概有 7 种常见的进程间的通信方式。

> 下面这部分总结参考了:《进程间通信 IPC (InterProcess Communication)》 这篇文章，推荐阅读，总结的非常不错。

1. **管道/匿名管道(Pipes)**：用于具有亲缘关系的父子进程间或者兄弟进程之间的通信。
2. **有名管道(Names Pipes)**：匿名管道由于没有名字，只能用于亲缘关系的进程间通信。为了克服这个缺点，提出了有名管道。有名管道严格遵循**先进先出(first in first out)**。有名管道以磁盘文件的方式存在，可以实现本机任意两个进程通信。

内容来源：csdn.net
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

3. **信号(Signal)**： 信号是一种比较复杂的通信方式，用于通知接收进程某个事件已经发生；
4. **消息队列(Message Queuing)**： 消息队列是消息的链表,具有特定的格式,存放在内存中并由消息队列标识符标识。管道和消息队列的通信数据都是先进先出的原则。与管道（无名管道：只存在于内存中的文件；命名管道：存在于实际的磁盘介质或者文件系统）不同的是消息队列存放在内核中，只有在内核重启(即，操作系统重启)或者显示地删除一个消息队列时，该消息队列才会被真正的删除。消息队列可以实现消息的随机查询,消息不一定要以先进先出的次序读取,也可以按消息的类型读取.比 FIFO 更有优势。**消息队列克服了信号承载信息量少，管道只能承载无格式字 节流以及缓冲区大小受限等缺。**
5. **信号量(Semaphores)**： 信号量是一个计数器，用于多进程对共享数据的访问，信号量的意图在于进程间同步。这种通信方式主要用于解决与同步相关的问题并避免竞争条件。
6. **共享内存(Shared memory)**： 使得多个进程可以访问同一块内存空间，不同进程可以及时看到对方进程中对共享内存中数据的更新。这种方式需要依靠某种同步操作，如互斥锁和信号量等。可以说这是最有用的进程间通信方式。
7. **套接字(Sockets)** : 此方法主要用于在客户端和服务器之间通过网络进行通信。套接字是支持 TCP/IP 的网络通信的基本操作单元，可以看做是不同主机之间的进程进行双向通信的端点，简单的说就是通信的两方的一种约定，用套接字中的相关函数来完成通信过程。

2.4 线程间的同步的方式

👮‍♂️ 面试官： 那线程间的同步的方式有哪些呢？

🙋‍♂️ 我： 线程同步是两个或多个共享关键资源的线程的并发执行。应该同步线程以避免关键的资源使用冲突。操作系统一般有下面三种线程同步的方式：

1. **互斥量(Mutex)**：采用互斥对象机制，只有拥有互斥对象的线程才有访问公共资源的权限。因为互斥对象只有一个，所以可以保证公共资源不会被多个线程同时访问。比如 Java 中的 synchronized 关键词和各种 Lock 都是这种机制。
2. **信号量(Semphares)**： 它允许同一时刻多个线程访问同一资源，但是需要控制同一时刻访问此资源的最大线程数量
3. **事件(Event)** :Wait/Notify：通过通知操作的方式来保持多线程同步，还可以方便的实现多线程优先级的比较操

2.5 进程的调度算法

👮‍♂️ 面试官： 你知道操作系统中进程的调度算法有哪些吗？

🙋‍♂️ 我： 嗯嗯！这个我们大学的时候学过，是一个很重要的知识点！

为了确定首先执行哪个进程以及最后执行哪个进程以实现最大 CPU 利用率，计算机科学家已经定义了一些算法，它们是：

- **先到先服务(FCFS)调度算法** : 从就绪队列中选择一个最先进入该队列的进程为之分配资源，使它立即执行并一直执行到完成或发生某事件而被阻塞放弃占用 CPU 时再重新调度。

内容来源：CSDN.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

- **短作业优先(SJF)的调度算法**：从就绪队列中选出一个估计运行时间最短的进程为之分配资源，使它立即执行并一直执行到完成或发生某事件而被阻塞放弃占用 CPU 时再重新调度。
- **时间片轮转调度算法**：时间片轮转调度是一种最古老，最简单，最公平且使用最广的算法，又称 RR(Round robin)调度。每个进程被分配一个时间段，称作它的时间片，即该进程允许运行的时间。
- **多级反馈队列调度算法**： 前面介绍的几种进程调度的算法都有一定的局限性。如**短进程优先的调度算法，仅照顾了短进程而忽略了长进程** 。多级反馈队列调度算法既能使高优先级的作业得到响应又能使短作业（进程）迅速完成。，因而它是目前**被公认的一种较好的进程调度算法**，UNIX 操作系统采取的便是这种调度算法。
- **优先级调度**： 为每个流程分配优先级，首先执行具有最高优先级的进程，依此类推。具有相同优先级的进程以 FCFS 方式执行。可以根据内存要求，时间要求或任何其他资源要求来确定优先级。

三 操作系统内存管理基础

3.1 内存管理介绍

👮‍♂️ **面试官: 操作系统的内存管理主要是做什么？**

🙋‍♂️ **我：** 操作系统的内存管理主要负责内存的分配与回收（malloc 函数：申请内存，free 函数：释放内存），另外地址转换也就是将逻辑地址转换成相应的物理地址等功能也是操作系统内存管理做的事情。

3.2 常见的几种内存管理机制

👮‍♂️ **面试官: 操作系统的内存管理机制了解吗？内存管理有哪几种方式？**

🙋‍♂️ **我：** 这个在学习操作系统的时候有了解过。

简单分为**连续分配管理方式**和**非连续分配管理方式**这两种。连续分配管理方式是指为一个用户程序分配一个连续的内存空间，常见的如 **块式管理** 。同样地，非连续分配管理方式允许一个程序使用的内存分布在离散或者说不相邻的内存中，常见的如**页式管理** 和 **段式管理**。

1. **块式管理**： 远古时代的计算机操系统的内存管理方式。将内存分为几个固定大小的块，每个块中只包含一个进程。如果程序运行需要内存的话，操作系统就分配给它一块，如果程序运行只需要很小的空间的话，分配的这块内存很大一部分几乎被浪费了。这些在每个块中未被利用的空间，我们称之为碎片。
2. **页式管理**： 把主存分为大小相等且固定的一页一页的形式，页较小，相对相比于块式管理的划分力度更大，提高了内存利用率，减少了碎片。页式管理通过页表对应逻辑地址和物理地址。
3. **段式管理**： 页式管理虽然提高了内存利用率，但是页式管理其中的页实际并无任何实际意义。 段式管理把主存分为一段段的，每一段的空间又要比一页的空间小很多 。但是，最重要的是段是有实际意义的，每个段定义了一组逻辑信息，例如,有主程序段 MAIN、子程序段 X、数据段 D 及栈段 S 等。 段式管理

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

通过段表对应逻辑地址和物理地址。

👮‍♂️ **面试官**： 回答的还不错！不过漏掉了一个很重要的 **段页式管理机制** 。段页式管理机制结合了段式管理和页式管理的优点。简单来说段页式管理机制就是把主存先分成若干段，每个段又分成若干页，也就是说 **段页式管理机制** 中段与段之间以及段的内部的都是离散的。

🙋‍♂️ **我**： 谢谢面试官！刚刚把这个给忘记了～

3.3 快表和多级页表

👮‍♂️ **面试官**： 页表管理机制中有两个很重要的概念：快表和多级页表，这两个东西分别解决了页表管理中很重要的两个问题。你给我简单介绍一下吧！

🙋‍♂️ **我**： 在分页内存管理中，很重要的两点是：

1. 虚拟地址到物理地址的转换要快。
2. 解决虚拟地址空间大，页表也会很大的问题。

快表

为了解决虚拟地址到物理地址的转换速度，操作系统在 **页表方案** 基础之上引入了 **快表** 来加速虚拟地址到物理地址的转换。我们可以把块表理解为一种特殊的高速缓冲存储器（Cache），其中的内容是页表的一部分或者全部内容。作为页表的 Cache，它的作用与页表相似，但是提高了访问速率。由于采用页表做地址转换，读写内存数据时 CPU 要访问两次主存。有了快表，有时只要访问一次高速缓冲存储器，一次主存，这样可加速查找并提高指令执行速度。

使用快表之后的地址转换流程是这样的：

1. 根据虚拟地址中的页号查快表；
2. 如果该页在快表中，直接从快表中读取相应的物理地址；
3. 如果该页不在快表中，就访问内存中的页表，再从页表中得到物理地址，同时将页表中的该映射表项添加到快表中；
4. 当快表填满后，又要登记新页时，就按照一定的淘汰策略淘汰掉快表中的一个页。

看完了之后你会发现快表和我们平时经常在我们开发的系统使用的缓存（比如 Redis）很像，的确是这样的，操作系统中的很多思想、很多经典的算法，你都可以在我们日常开发使用的各种工具或者框架中找到它们的影子。

多级页表

引入多级页表的主要目的是为了避免把全部页表一直放在内存中占用过多空间，特别是那些根本就不需要的页表就不需要保留在内存中。多级页表属于时间换空间的典型场景，具体可以查看下面这篇文章

内容来源：csdn.net
作者昵称：kuangd_1992
作者主页：https://blog.csdn.net/kuangd_1992

- 多级页表如何节约内存：https://www.polarxiong.com/archives/多级页表如何节约内存.html

总结

为了提高内存的空间性能，提出了多级页表的概念；但是提到空间性能是以浪费时间性能为基础的，因此为了补充损失的时间性能，提出了快表（即 TLB）的概念。 不论是快表还是多级页表实际上都利用到了程序的局部性原理，局部性原理在后面的虚拟内存这部分会介绍到。

3.4 分页机制和分段机制的共同点和区别

面试官： 分页机制和分段机制有哪些共同点和区别呢?

我：

1. **共同点：**
    - 分页机制和分段机制都是为了提高内存利用率，较少内存碎片。
    - 页和段都是离散存储的，所以两者都是离散分配内存的方式。但是，每个页和段中的内存是连续的。
2. **区别：**
    - 页的大小是固定的，由操作系统决定；而段的大小不固定，取决于我们当前运行的程序。
    - 分页仅仅是为了满足操作系统内存管理的需求，而段是逻辑信息的单位，在程序中可以体现为代码段，数据段，能够更好满足用户的需要。

3.5 逻辑(虚拟)地址和物理地址

面试官： 你刚刚还提到了**逻辑地址和物理地址**这两个概念，我不太清楚，你能为我解释一下不?

我： em...好的嘛！我们编程一般只有可能和逻辑地址打交道，比如在 C 语言中，指针里面存储的数值就可以理解成为内存里的一个地址，这个地址也就是我们说的逻辑地址，逻辑地址由操作系统决定。物理地址指的是真实物理内存中地址，更具体一点来说就是内存地址寄存器中的地址。物理地址是内存单元真正的地址。

3.6 CPU 寻址了解吗?为什么需要虚拟地址空间?

面试官： CPU 寻址了解吗?为什么需要虚拟地址空间?

我： 这部分我真不清楚！

于是面试完之后我默默去查阅了相关文档！留下了没有技术的泪水。。。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

> 这部分内容参考了 Microsoft 官网的介绍，地址：https://msdn.microsoft.com/zh-cn/library/windows/hardware/hh439648(v=vs.85).aspx

现代处理器使用的是一种称为 **虚拟寻址(Virtual Addressing)** 的寻址方式。**使用虚拟寻址，CPU 需要将虚拟地址翻译成物理地址，这样才能访问到真实的物理内存。** 实际上完成虚拟地址转换为物理地址转换的硬件是 CPU 中含有一个被称为 **内存管理单元（Memory Management Unit, MMU）** 的硬件。如下图所示：

[外链图片转存失败,源站可能有防盗链机制,建议将图片保存下来直接上传(img-fW9Lbd0k-1685369448150)(C:\Users\kd\Downloads\baguwen-wiki-master\docs\img\69f5eca6.png)]

为什么要有虚拟地址空间呢?

先从没有虚拟地址空间的时候说起吧！没有虚拟地址空间的时候，**程序都是直接访问和操作的都是物理内存** 。但是这样有什么问题呢?

1. 用户程序可以访问任意内存，寻址内存的每个字节，这样就很容易（有意或者无意）破坏操作系统，造成操作系统崩溃。
2. 想要同时运行多个程序特别困难，比如你想同时运行一个微信和一个 QQ 音乐都不行。为什么呢？举个简单的例子：微信在运行的时候给内存地址 1xxx 赋值后，QQ 音乐也同样给内存地址 1xxx 赋值，那么 QQ 音乐对内存的赋值就会覆盖微信之前所赋的值，这就造成了微信这个程序就会崩溃。

总结来说：如果直接把物理地址暴露出来的话会带来严重问题，比如可能对操作系统造成伤害以及给同时运行多个程序造成困难。

通过虚拟地址访问内存有以下优势：

- 程序可以使用一系列相邻的虚拟地址来访问物理内存中不相邻的大内存缓冲区。
- 程序可以使用一系列虚拟地址来访问大于可用物理内存的内存缓冲区。当物理内存的供应量变小时，内存管理器会将物理内存页（通常大小为 4 KB）保存到磁盘文件。数据或代码页会根据需要在物理内存与磁盘之间移动。
- 不同进程使用的虚拟地址彼此隔离。一个进程中的代码无法更改正在由另一进程或操作系统使用的物理内存。

四 虚拟内存

4.1 什么是虚拟内存(Virtual Memory)?

👮 **面试官** ： 再问你一个常识性的问题！**什么是虚拟内存(Virtual Memory)?**

🙋 **我** ： 这个在我们平时使用电脑特别是 Windows 系统的时候太常见了。很多时候我们使用点开了很多占内存的软件，这些软件占用的内存可能已经远远超出了我们电脑本身具有的物理内存。**为什么可以这样呢?** 正是因为 **虚拟内存** 的存在，通过 **虚拟内存** 可以让程序可以拥有超过系统物理内存大小的可用内存空间。另外，**虚拟内存为每个进程提供了一个一致的、私有的地址空间，它让每个进程产生了一种自己在独享主存的错觉（每个进程拥有一片连续完整的内存空间）**。这样会更加有效地管理内存并减少出错。

版权声明：本博客
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

虚拟内存是计算机系统内存管理的一种技术，我们可以手动设置自己电脑的虚拟内存。不要单纯认为虚拟内存只是“使用硬盘空间来扩展内存“的技术。**虚拟内存的重要意义是它定义了一个连续的虚拟地址空间**，并且 **把内存扩展到硬盘空间**。推荐阅读：《虚拟内存的那点事儿》

维基百科中有几句话是这样介绍虚拟内存的。

> **虚拟内存** 使得应用程序认为它拥有连续的可用的内存（一个连续完整的地址空间），而实际上，它通常是被分隔成多个物理内存碎片，还有部分暂时存储在外部磁盘存储器上，在需要时进行数据交换。与没有使用虚拟内存技术的系统相比，使用这种技术的系统使得大型程序的编写变得更容易，对真正的物理内存（例如 RAM）的使用也更有效率。目前，大多数操作系统都使用了虚拟内存，如 Windows 家族的“虚拟内存”；Linux 的“交换空间”等。
> From:https://zh.wikipedia.org/wiki/虚拟内存

4.2 局部性原理

👮 **面试官**： 要想更好地理解虚拟内存技术，必须要知道计算机中著名的**局部性原理**。另外，局部性原理既适用于程序结构，也适用于数据结构，是非常重要的一个概念。

🙋 **我**： 局部性原理是虚拟内存技术的基础，正是因为程序运行具有局部性原理，才可以只装入部分程序到内存就开始运行。

> 以下内容摘自《计算机操作系统教程》 第 4 章存储器管理。

早在 1968 年的时候，就有人指出我们的程序在执行的时候往往呈现局部性规律，也就是说在某个较短的时间段内，程序执行局限于某一小部分，程序访问的存储空间也局限于某个区域。

局部性原理表现在以下两个方面：

1. **时间局部性**： 如果程序中的某条指令一旦执行，不久以后该指令可能再次执行；如果某数据被访问过，不久以后该数据可能再次被访问。产生时间局部性的典型原因，是由于在程序中存在着大量的循环操作。
2. **空间局部性**： 一旦程序访问了某个存储单元，在不久之后，其附近的存储单元也将被访问，即程序在一段时间内所访问的地址，可能集中在一定的范围之内，这是因为指令通常是顺序存放、顺序执行的，数据也一般是以向量、数组、表等形式簇聚存储的。

时间局部性是通过将近来使用的指令和数据保存到高速缓存存储器中，并使用高速缓存的层次结构实现。空间局部性通常是使用较大的高速缓存，并将预取机制集成到高速缓存控制逻辑中实现。虚拟内存技术实际上就是建立了 “内存—外存”的两级存储器的结构，利用局部性原理实现高速缓存。

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

4.3 虚拟存储器

👮 **面试官**： 都说了虚拟内存了。你再讲讲**虚拟存储器**把！

🙋 **我：**

> 这部分内容来自：王道考研操作系统知识点整理。

基于局部性原理，在程序装入时，可以将程序的一部分装入内存，而将其他部分留在外存，就可以启动程序执行。由于外存往往比内存大很多，所以我们运行的软件的内存大小实际上是可以比计算机系统实际的内存大小大的。在程序执行过程中，当所访问的信息不在内存时，由操作系统将所需要的部分调入内存，然后继续执行程序。另一方面，操作系统将内存中暂时不使用的内容换到外存上，从而腾出空间存放将要调入内存的信息。这样，计算机好像为用户提供了一个比实际内存大的多的存储器——**虚拟存储器**。

实际上，我觉得虚拟内存同样是一种时间换空间的策略，你用 CPU 的计算时间，页的调入调出花费的时间，换来了一个虚拟的更大的空间来支持程序的运行。不得不感叹，程序世界几乎不是时间换空间就是空间换时间。

4.4 虚拟内存的技术实现

👮‍♂️ **面试官： 虚拟内存技术的实现呢？**

🙋 **我： 虚拟内存的实现需要建立在离散分配的内存管理方式的基础上。** 虚拟内存的实现有以下三种方式：

1. **请求分页存储管理**： 建立在分页管理之上，为了支持虚拟存储器功能而增加了请求调页功能和页面置换功能。请求分页是目前最常用的一种实现虚拟存储器的方法。请求分页存储管理系统中，在作业开始运行之前，仅装入当前要执行的部分段即可运行。假如在作业运行的过程中发现要访问的页面不在内存，则由处理器通知操作系统按照对应的页面置换算法将相应的页面调入到主存，同时操作系统也可以将暂时不用的页面置换到外存中。
2. **请求分段存储管理**： 建立在分段存储管理之上，增加了请求调段功能、分段置换功能。请求分段储存管理方式就如同请求分页储存管理方式一样，在作业开始运行之前，仅装入当前要执行的部分段即可运行；在执行过程中，可使用请求调入中断动态装入要访问但又不在内存的程序段；当内存空间已满，而又需要装入新的段时，根据置换功能适当调出某个段，以便腾出空间而装入新的段。
3. **请求段页式存储管理**

这里多说一下？很多人容易搞混请求分页与分页存储管理，两者有何不同呢？

请求分页存储管理建立在分页管理之上。他们的根本区别是是否将程序全部所需的全部地址空间都装入主存，这也是请求分页存储管理可以提供虚拟内存的原因，我们在上面已经分析过了。

它们之间的根本区别在于是否将一作业的全部地址空间同时装入主存。请求分页存储管理不要求将作业全部地址空间同时装入主存。基于这一点，请求分页存储管理可以提供虚存，而分页存储管理却不能提供虚存。

版权声明：本文为博主原创文章，遵循 CC 4.0 BY-SA 版权协议，转载请附上原文出处链接和本声明。
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

不管是上面那种实现方式，我们一般都需要：

1. 一定容量的内存和外存：在载入程序的时候，只需要将程序的一部分装入内存，而将其他部分留在外存，然后程序就可以执行了；
2. **缺页中断**：如果**需执行的指令或访问的数据尚未在内存**（称为缺页或缺段），则由处理器通知操作系统将相应的页面或段**调入到内存**，然后继续执行程序；
3. **虚拟地址空间**： 逻辑地址到物理地址的变换。

4.5 页面置换算法

👮‍♂️ **面试官**： 虚拟内存管理很重要的一个概念就是页面置换算法。那你说一下 **页面置换算法的作用?常见的页面置换算法有哪些?**

🙋‍♀️ **我**：

> 这个题目经常作为笔试题出现，网上已经给出了很不错的回答，我这里只是总结整理了一下。

地址映射过程中，若在页面中发现所要访问的页面不在内存中，则发生缺页中断 。

> **缺页中断** 就是要访问的**页**不在主存，需要操作系统将其调入主存后再进行访问。 在这个时候，被内存映射的文件实际上成了一个分页交换文件。

当发生缺页中断时，如果当前内存中并没有空闲的页面，操作系统就必须在内存选择一个页面将其移出内存，以便为即将调入的页面让出空间。用来选择淘汰哪一页的规则叫做页面置换算法，我们可以把页面置换算法看成是淘汰页面的规则。

- **OPT 页面置换算法（最佳页面置换算法）**： 最佳(Optimal, OPT)置换算法所选择的被淘汰页面将是以后永不使用的，或者是在最长时间内不再被访问的页面,这样可以保证获得最低的缺页率。但由于人们目前无法预知进程在内存下的若干页面中哪个是未来最长时间内不再被访问的，因而该算法无法实现。一般作为衡量其他置换算法的方法。
- **FIFO（First In First Out） 页面置换算法（先进先出页面置换算法）**：总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面进行淘汰。
- **LRU （Least Currently Used）页面置换算法（最近最久未使用页面置换算法）**： LRU算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间 T，当须淘汰一个页面时，选择现有页面中其 T 值最大的，即最近最久未使用的页面予以淘汰。
- **LFU （Least Frequently Used）页面置换算法（最少使用页面置换算法）**：该置换算法选择在之前时期使用最少的页面作为淘汰页。

Reference

- 《计算机操作系统—汤小丹》第四版
- 《深入理解计算机系统》

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992

- https://zh.wikipedia.org/wiki/输入输出内存管理单元
- https://baike.baidu.com/item/快表/19781679
- https://www.jianshu.com/p/1d47ed0b46d5
- https://www.studytonight.com/operating-system
- https://www.geeksforgeeks.org/interprocess-communication-methods/
- https://juejin.im/post/59f8691b51882534af254317
  OPT 页面置换算法（最佳页面置换算法）： 最佳(Optimal, OPT)置换算法所选择的被淘汰页面将是以后永不使用的，或者是在最长时间内不再被访问的页面,这样可以保证获得最低的缺页率。但由于人们目前无法预知进程在内存下的若干页面中哪个是未来最长时间内不再被访问的，因而该算法无法实现。一般作为衡量其他置换算法的方法。
- **FIFO（First In First Out） 页面置换算法（先进先出页面置换算法）**：总是淘汰最先进入内存的页面，即选择在内存中驻留时间最久的页面进行淘汰。
- **LRU （Least Currently Used）页面置换算法（最近最久未使用页面置换算法）**： LRU算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间 T，当须淘汰一个页面时，选择现有页面中其 T 值最大的，即最近最久未使用的页面予以淘汰。
- **LFU （Least Frequently Used）页面置换算法（最少使用页面置换算法）**：该置换算法选择在之前时期使用最少的页面作为淘汰页。

Reference

- 《计算机操作系统—汤小丹》第四版
- 《深入理解计算机系统》
- https://zh.wikipedia.org/wiki/输入输出内存管理单元
- https://baike.baidu.com/item/快表/19781679
- https://www.jianshu.com/p/1d47ed0b46d5
- https://www.studytonight.com/operating-system
- https://www.geeksforgeeks.org/interprocess-communication-methods/
- https://juejin.im/post/59f8691b51882534af254317
- 王道考研操作系统知识点整理： https://wizardforcel.gitbooks.io/wangdaokaoyan-os/content/13.html

内容来源：csdn.net
作者昵称：kuangd_1992
原文链接：https://blog.csdn.net/kuangd_1992/article/details/130937703
作者主页：https://blog.csdn.net/kuangd_1992